

Công nghệ phần mềm

Chương 05: Lập trình



Giảng viên: Lê Thị Hoàng Anh

Email: anh1th@nuce.edu.vn



Nội dung

1. Các đặc trưng ngôn ngữ lập trình

- Đặc trưng tâm lý của ngôn ngữ lập trình
- Mô hình cú pháp và ngữ nghĩa
- Hướng quan điểm kỹ nghệ
- Việc chọn ngôn ngữ
- Ngôn ngữ lập trình và kỹ nghệ phần mềm

2. Công cụ và môi trường phát triển tích hợp

3. Phong cách lập trình



Các đặc trưng ngôn ngữ lập trình

- Ngôn ngữ lập trình là phương pháp để liên lạc giữa con người và máy tính.
- Lập trình:
 - Là một hoạt động của con người
 - Là sự liên lạc thông qua ngôn ngữ lập trình
 - Là một bước cốt lõi trong tiến trình kỹ nghệ phần mềm



Các đặc trưng ngôn ngữ lập trình

- Đặc trưng tâm lý của ngôn ngữ lập trình:
 - Tính đồng đều
 - Tính mơ hồ
 - Tính gọn gàng
 - Tính cục bộ
 - Tính tuyến tính



Các đặc trưng ngôn ngữ lập trình

- Mô hình cú pháp và ngữ nghĩa:
 - Mô tả của một ngôn ngữ lập trình thường được chia thành hai thành phần:
 - Cú pháp (hình thức)
 - Và ngữ nghĩa (ý nghĩa)
 - Cú pháp: là khái niệm chỉ ra bản thân câu lệnh có hợp lệ với cấu trúc và ngữ pháp của ngôn ngữ hay không?
 - Ngữ nghĩa: là về việc liệu câu lệnh đã viết có ý nghĩa hợp lệ hay không?



Các đặc trưng ngôn ngữ lập trình

- Mô hình lập trình:
 - Mô hình lập trình là một cách phân loại ngôn ngữ lập trình dựa trên tính năng của chúng.
 - Một ngôn ngữ có thể hỗ trợ nhiều mô hình lập trình.
 - Một số mô hình chủ yếu quan tâm đến sự thực thi của chương trình, các mô hình khác lại chủ yếu quan tâm đến cách tổ chức mã code, tuy nhiên những mô hình khác lại chỉ quan tâm đến phong cách của cú pháp và ngữ pháp.



Các đặc trưng ngôn ngữ lập trình

- Mô hình lập trình (tiếp)
 - Các mô hình lập trình phổ biến bao gồm:
 - Lập trình mệnh lệnh: Là mẫu hình lập trình sử dụng một chuỗi các câu lệnh để thay đổi trạng thái của chương trình. Bao gồm như:
 - Lập trình hướng thủ tục thực hiện nhóm các câu lệnh trong các thủ tục/các hàm.
 - Lập trình hướng đối tượng thực hiện code để tạo ra các đối tượng trừu tượng hóa các đối tượng trong bài toán thực tế.
 - Lập trình khai báo: Trong đó các lập trình viên khai báo các thuộc tính của kết quả mong muốn, nhưng không thực hiện tính toán nó.



Các đặc trưng ngôn ngữ lập trình

- Mô hình lập trình (tiếp)
 - Lập trình khai báo bao gồm như:
 - Lập trình hướng chức năng: Kết quả mong muốn được khai báo như dãy giá trị của chức năng các ứng dụng.
 - Lập trình logic: Dựa trên logic toán trong các mối quan hệ và các suy luận.
 - Lập trình tính toán: Trong đó kết quả mong muốn được khai báo là giải pháp của một bài toán tối ưu.
 - Lập trình phản ứng(reactive): Trong đó kết quả mong muốn được khai báo với các luồng dữ liệu và sự lan truyền của sự thay đổi.



Các đặc trưng ngôn ngữ lập trình

- Ví dụ: Duyệt danh sách đưa ra số chẵn
 - Lập trình mệnh lệnh:

```
List collection = new List { 1, 2, 3, 4, 5 };  
List results = new List();  
foreach(var num in collection)  
{  
    if (num % 2 != 0)  
        results.Add(num);  
}
```

- Lập trình khai báo:

```
List collection = new List { 1, 2, 3, 4, 5 };  
var results = collection.Where( num => num % 2 != 0);
```



Các đặc trưng ngôn ngữ lập trình

- Hướng quan điểm kỹ nghệ:
 - Cách nhìn kỹ nghệ phần mềm về các đặc trưng của ngôn ngữ lập trình tập trung vào nhu cầu xác định dự án phát triển phần mềm riêng.
 - Mặc dầu người ta vẫn cần các yêu cầu riêng cho chương trình gốc, vẫn có thể thiết lập được một tập hợp tổng quát những đặc trưng kỹ nghệ:



Các đặc trưng ngôn ngữ lập trình

- Hướng quan điểm kỹ nghệ: (tiếp)
 - Các đặc trưng kỹ nghệ:
 1. Dễ dịch thiết kế sang chương trình
 2. Có trình biên dịch hiệu quả
 3. Khả chuyển chương trình gốc
 4. Có sẵn công cụ phát triển
 5. Dễ bảo trì



Các đặc trưng ngôn ngữ lập trình

- Tính khả chuyển chương trình gốc:
 - Là một đặc trưng của ngôn ngữ lập trình có thể được hiểu theo ba cách khác nhau:
 - Chương trình gốc có thể chuyển đổi giữa các bộ xử lý và các trình biên dịch khác nhau với rất ít hoặc không sửa đổi gì.
 - Chương trình gốc vẫn không thay đổi ngay cả khi môi trường của nó thay đổi.
 - Chương trình gốc có thể được tích hợp vào trong các bộ trình phần mềm (các trình IDE) khác nhau với ít hay không cần thay đổi gì vì các đặc trưng của ngôn ngữ lập trình.



Các đặc trưng ngôn ngữ lập trình

- Tính sẵn có của công cụ phát triển:
 - Có thể làm giảm bớt thời gian cần để sinh ra chương trình gốc và có thể cải thiện chất lượng của chương trình.
 - Nhiều ngôn ngữ lập trình có thể cần tới một loạt các công cụ như:
 - Trình biên dịch gỡ lỗi trợ giúp chương trình gốc
 - Các tiện nghi soạn thảo có sẵn
 - Các công cụ kiểm soát chương trình gốc
 - Thư viện chương trình con mở rộng
 - Công cụ kỹ nghệ ngược và các công cụ khác tạo nên một “môi trường phát triển phần mềm”



Các đặc trưng ngôn ngữ lập trình

- Việc chọn ngôn ngữ:
 - Việc lựa chọn ngôn ngữ lập trình cho một dự án riêng phải tính đến cả đặc trưng kỹ nghệ và tâm lý.
 - Về mặt kỹ nghệ: Khi phải lựa chọn một NNLT nên bắt đầu từ miền vấn đề của dự án, hiệu suất, thư viện hỗ trợ, mức độ phổ biến và cộng đồng hỗ trợ ...
 - Về mặt tâm lý: Dựa vào các kỹ năng và kinh nghiệm trong nhóm đã có sẵn, quen thuộc và đã thành công trong quá khứ, ít thay đổi và không chuyển sang cái mới.
 - Tuy nhiên trong doanh nghiệp phải thừa nhận về sự kháng cự tâm lý



Các đặc trưng ngôn ngữ lập trình

- Ngôn ngữ lập trình và kỹ nghệ phần mềm:
 - Bất kể dự án phát triển theo mô hình phát triển phần mềm nào thì ngôn ngữ lập trình sẽ có tác động tới việc vạch kế hoạch dự án, phân tích, thiết kế, lập trình, kiểm thử và bảo trì.
 - Ví dụ như:
 - Ngôn ngữ lập trình cần phù hợp với:
 - Cấu trúc dữ liệu phức tạp của dự án và thiết kế dữ liệu
 - Nếu dự án cần hiệu năng cao và lập trình thời gian thực cần chọn ngôn ngữ lập trình hỗ trợ thời gian thực.
 - Nếu chọn kiểm thử tích hợp thì cần ngôn ngữ lập trình hỗ trợ tính modul ...



Nội dung

→ 2. Công cụ và môi trường phát triển tích hợp

- Môi trường phát triển tích hợp
- Công cụ sinh mã nguồn từ thiết kế
- Công cụ quản lý mã nguồn
- Công cụ lập tài liệu chương trình

3. Phong cách lập trình



Môi trường phát triển tích hợp

- Còn được gọi là *IDE*, *môi trường thiết kế hợp nhất* hay *môi trường gỡ lỗi hợp nhất*.
- Là một loại phần mềm máy tính có công dụng giúp đỡ các lập trình viên trong việc phát triển phần mềm.
- Phân loại theo số lượng các ngôn ngữ được hỗ trợ:
 - Môi trường phát triển hợp nhất một ngôn ngữ
 - Môi trường phát triển hợp nhất đa ngôn ngữ



Môi trường phát triển tích hợp

- Môi trường phát triển tích hợp bao gồm:
 - Trình soạn thảo mã nguồn
 - Trình biên dịch hoặc trình thông dịch
 - Công cụ xây dựng tự động
 - Biên dịch mã nguồn (hoặc thông dịch)
 - Thực hiện liên kết (linking)
 - Và có thể chạy chương trình một cách tự động
 - Trình gỡ lỗi
 - Ngoài ra còn có: Hệ thống quản lý phiên bản, lược đồ phân cấp lớp, trình quản lý đối tượng

...



Môi trường phát triển tích hợp

- Lịch sử phát triển:
 - Với thế hệ máy tính đời đầu, việc viết chương trình gắn liền với việc phải thay đổi cấu trúc, linh kiện... của máy. Dữ liệu vào bằng các tấm thẻ đục lỗ
 - Sau đó, khi màn hình máy tính ra đời thì IDE mới ra đời. BASIC là ngôn ngữ đầu tiên có IDE cho riêng mình.
 - Hoàn toàn dựa trên ký tự.
 - Không có các tính năng giao diện đồ họa như hiện nay.



Môi trường phát triển tích hợp

- Lịch sử phát triển (tiếp):
 - Khi có hệ thống cửa sổ Microsoft Windows và X11 thì các IDE mới có giao diện đồ họa và có thêm ngày càng nhiều các chức năng như ngày nay.
- Cần phân biệt Môi trường phát triển tích hợp (IDE) với Bộ công cụ phát triển phần mềm (SDK) hay công cụ soạn thảo văn bản (như vi, emacs ...)



Công cụ sinh mã nguồn từ thiết kế

- Nguyên nhân cần sinh mã tự động:
 - Do nhu cầu chuyển đổi số cần phát triển sản phẩm phần mềm ngày càng tăng.
 - Đội ngũ nhân lực CNTT thiếu và yếu: Theo TopDev tình hình nguồn nhân lực:
 - Năm 2020, nhu cầu cần 400.000 nhân lực, thiếu 100.000 nhân lực
 - Năm 2021, nhu cầu cần khoảng 500.000 nhân lực
 - Nắm bắt được nhu cầu, trên thị trường bùng nổ các nền tảng và công cụ no-code/low-code với sự hỗ trợ của trí tuệ nhân tạo AI
 - Cần tiết kiệm chi phí cho dự án



Công cụ sinh mã nguồn từ thiết kế

- Năm 2021, NC/LC được dự báo thuộc nhóm top 5 về xu hướng CĐS trên thế giới
- Đội ngũ *citizen developer* (“LTV nhân dân”) trong xã hội là rất lớn, có tiềm năng tạo đột phá CĐS rất nhanh cho xã hội
- Nhờ có AI, nền tảng NC/LC (nền tảng lập trình thể hệ thứ 4) tự động sinh mã chương trình từ câu lệnh bằng ngôn ngữ tự nhiên hay giao diện đồ họa kéo thả để “lắp ghép phần mềm”



Công cụ sinh mã nguồn từ thiết kế

Hype Cycle for the Digital Workplace, 2020



gartner.com/SmarterWithGartner

Source: Gartner
© 2020 Gartner, Inc. and/or its affiliates. All rights reserved. Gartner and Hype Cycle are registered trademarks of Gartner, Inc. and its affiliates in the U.S.

Gartner

Citizen developer bắt đầu phổ biến vào năm 2022
(theo Gartner)



Công cụ sinh mã nguồn từ thiết kế

- Lợi ích từ sinh mã nguồn tự động:
 - Hữu ích cho việc phát triển các sản phẩm thử nghiệm và hoàn thiện ý tưởng, xác định thị trường...
 - Giảm chi phí dự án và
 - Giảm sự thiếu hụt nguồn nhân lực lập trình viên chuyên nghiệp trong giai đoạn lập trình và thiết kế *tạo nguyên mẫu* cho hệ thống.



Công cụ sinh mã nguồn từ thiết kế

- Sinh mã nguồn tự động chỉ phù hợp với:
 - Phù hợp với các ứng dụng, module, hay phân hệ có nghiệp vụ xử lý đơn giản.
 - Các thao tác có thể chuẩn hóa. Ví dụ như:
 - Các nghiệp vụ CRUD dữ liệu (Create Request/Read Update Delete)
 - Các modul, chức năng tổng hợp, xử lý chung tổng quát, hay đóng gói được.



Công cụ sinh mã nguồn từ thiết kế

- Dự báo thị trường của nền tảng low-code toàn cầu:
 - Năm 2021 tăng 23%
 - Năm 2023, 50% doanh nghiệp vừa và nhỏ xem low-code là nền tảng chiến lược
 - Năm 2024, 65% việc phát triển ứng dụng sẽ thực hiện bằng low-code
 - Năm 2030, đội ngũ LTV nhân dân trong các công ty lớn sẽ nhiều gấp 4 lần số lượng lập trình viên chuyên nghiệp ở các công ty đó.



Công cụ sinh mã nguồn từ thiết kế

- Giới thiệu một số sản phẩm sinh code tự động trên thị trường Việt Nam:
 - tGenCode của Tavico:
 - Sản phẩm tự sinh code cho các phần mềm quản lý kho, danh mục.
 - Phát triển trên nền tảng .Net và CSDL SQL Server
 - ATC PRO:
 - Sản phẩm dành cho sinh viên và các nhà phát triển web, tự sinh code kết nối với CSDL mô hình 3 layer và 3 tier, tự sinh code các query và các procedure.
 - Nền tảng C# và SQL Server



Công cụ sinh mã nguồn từ thiết kế

- Demo cho sinh viên sinh code tự động từ bản thiết kế biểu đồ lớp (sẽ hướng dẫn kỹ hơn trong môn học phần đồ án môn học)



Công cụ quản lý mã nguồn

- Đăng ký tài khoản và tìm hiểu thêm về công cụ quản lý mã nguồn Github
<https://github.com/>



Công cụ lập tài liệu chương trình

- Tài liệu chương trình là gì?
 - Là tài liệu bên trong của chương trình gốc.
 - Bắt đầu với việc:
 - Chọn lựa các tên gọi định danh (biến và nhãn),
 - Vị trí và thành phần của việc chú thích
 - Và cách tổ chức trực quan của chương trình
 - Trong đó có nhiều hướng dẫn cho việc viết lời chú thích bao gồm:
 - Chú thích mở đầu: README/Javadoc ...
 - Và chú thích chức năng: Trong các modul mã code



Công cụ lập tài liệu chương trình

- Readme là gì?
 - Là một tệp văn bản (.txt hoặc .md) hoạt động như tài liệu cho một dự án hoặc một thư mục code.
 - Nó thường là phần documentation và landing page dễ thấy nhất trong các dự án mã nguồn mở. (được up trên GitHub)
 - README được viết hoa toàn bộ để thu hút sự chú ý của người đọc sẽ đọc nó đầu tiên.



Công cụ lập tài liệu chương trình

- Javadoc là gì?
 - Là một công cụ đi với JDK và nó được sử dụng để tạo java code documentation trong định dạng HTML từ chương trình gốc của java mà đã được yêu cầu build tài liệu documentation trong một định dạng cho trước.
 - Đây là Documentation Comment được công cụ JDK javadoc sử dụng khi tự động chuẩn bị document đã tạo.



Công cụ lập tài liệu chương trình

- Nên để những gì trong chú thích mở đầu README:
 - Đặt tên cho project (hoặc cái gì đó mà dự án hướng tới)
 - Giới thiệu hoặc tóm tắt:
 - Nên viết ngắn trong vòng 2 đến 3 dòng
 - Giải thích dự án làm gì? và viết cho ai?
 - Điều kiện tiên quyết:
 - Ngay sau phần giới thiệu nên đặt tiêu đề để liệt kê những kiến thức hoặc công cụ tiên quyết mà bất cứ ai muốn sử dụng dự án cần trước khi bắt đầu



Công cụ lập tài liệu chương trình

- Cách cài đặt/hướng dẫn sử dụng nếu cần
- Cách góp ý kiến
- Thêm contributors
- Thêm nguồn dẫn chứng
- Thông tin liên lạc
- Thông tin bản quyền
- Ngoài ra có thể thêm logo, badges, shields...



Công cụ lập tài liệu chương trình

- Cách viết chú thích mở đầu (Javadoc):
 - Để tạo công cụ Javadoc, các nhà phát triển đã định nghĩa Doclet API là cấu trúc interface, class mà chương trình chuyển đổi tài liệu cần cài đặt.
 - Chương trình chuyển đổi tài liệu được gọi là doclet, là tài liệu HTML dùng để xác định nội dung từ phần chú thích tài liệu trong mã nguồn Java.
 - Mặc định, Javadoc cung cấp doclet tiêu chuẩn, nhưng bạn có thể tùy chỉnh doclet này



Công cụ lập tài liệu chương trình

- Một số thẻ có thể đặt trong phần document

Thẻ	Miêu tả	Cú pháp
@author	Thêm author của một lớp	@author name-text
{@code}	Hiển thị text trong code font mà không thông dịch text như là các thẻ HTML markup hoặc javadoc được lồng.	{@code text}
{@docRoot}	Biểu diễn relative path tới thư mục root của tài liệu từ các trang được tạo	{@docRoot}
@deprecated	Thêm một comment chỉ dẫn rằng API này không còn được sử dụng nữa	@deprecated deprecated-text
@exception	Thêm một Throws subheading tới documentation được tạo, với tên lớp và text mô tả	@exception class-name Miêu tả
{@inheritDoc}	Kế thừa một comment từ lớp có thể kế thừa gần nhất hoặc interface có thể triển khai	Kế thừa một comment từ một lớp cha trung gian.



Công cụ lập tài liệu chương trình

<code>{@link}</code>	Chèn một in-line link với nhãn text có thể nhìn thấy mà chỉ tới Documentation cho package, lớp hoặc tên member đã xác định của một lớp tham chiếu	<code>{@link package.class#member label}</code>
<code>{@linkplain}</code>	Giống <code>{@link}</code> , ngoại trừ label của link được hiển thị ở dạng plain text thay vì code font.	<code>{@linkplain package.class#member label}</code>
<code>@param</code>	Thêm một tham số với parameter-name đã xác định được theo sau bởi description đã xác định tới khu vực "Parameters".	<code>@param parameter-name Miêu tả</code>
<code>@return</code>	Thêm một khu vực "Returns" với Miêu tả	<code>@return Miêu tả</code>
<code>@see</code>	Thêm một "See Also" heading với một link hoặc text mà chỉ tới reference	<code>@see reference</code>



Công cụ lập tài liệu chương trình

@serial	Được sử dụng trong doc comment cho một trường có thể xếp thứ tự.	@serial field-description include exclude
@serialData	Thu thập thông tin dữ liệu được viết bởi phương thức writeObject() hoặc writeExternal()	@serialData data-Miêu tả
@serialField	Thu thập thông tin một thành phần ObjectOutputStreamField	@serialField field-name field-type field-Miêu tả
@since	Thêm một "Since" heading với since-text đã xác định tới Documentation đã tạo	@since release
@throws	Thẻ @throws và @exception là như nhau	@throws class-name Miêu tả
{@value}	Khi {@value} được sử dụng trong doc comment của một trường static, nó hiển thị giá trị của hằng số đó	{@value package.class#field}
@version	Thêm một "Version" subheading với version-text đã xác định tới Documentation đã tạo khi tùy chọn -version được sử dụng	@version version-text



Công cụ lập tài liệu chương trình

- Ví dụ:

```
* Chương trình AddNum triển khai một ứng dụng mà
* cộng hai số integer đã cho và in chúng
* the output on the screen.
* <p>
* <b>Note:</b> Việc cung cấp comment thích hợp trong chương trình giúp nó trở nên
* thân thiện với người dùng hơn.
*
* @author Zara Ali
* @version 1.0
* @since 2014-03-31
*/
public class AddNum {
    /**
     * Phương thức này được sử dụng để cộng hai số. Đây là
     * một form đơn giản nhất của phương thức, để minh họa
     * các sử dụng của các javadoc Tags.
     * @param numA Đây là tham số đầu tiên của phương thức addNum
     * @param numB Đây là tham số thứ hai của phương thức addNum
     * @return int Trả về tổng của numA và numB.
     */
    public int addNum(int numA, int numB) {
        return numA + numB;
    } /**
     * Đây là phương thức chính mà sử dụng phương thức addNum.
     * @param args Unused.
     * @return Nothing.
     * @exception IOException On input error.
     * @see IOException
     */
    public static void main(String args[]) throws IOException
    {
        AddNum obj = new AddNum();
        int sum = obj.addNum(10, 20);    System.out.println("Tổng của 10 và 20 là :")
    }
}
```



Công cụ lập tài liệu chương trình

- VD

← → ↺ ⓘ File | C:/Users/anhlt/OneDrive/Documents/NetBeansProjects/test/dist/javadoc/test/Test.html

PACKAGE **CLASS** USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

Package test

Class Test

java.lang.Object[Ⓓ]
test.Test

public class Test
extends Object[Ⓓ]

Chương trình AddNum triển khai một ứng dụng mà cộng hai số integer đã cho và in chung the output on the screen.

Note: Việc cung cấp comment thích hợp trong chương trình giúp nó trở nên thân thiện với người dùng hơn.

Since:
2014-03-31

Version:
1.0

Author:
Zara Ali

Constructor Summary

Constructors

Constructor	Description
Test()	

Method Summary

All Methods	Static Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description	
int	addNum(int numA, int numB)	Phương thức này được sử dụng để cộng hai số.	
static void	main(String [Ⓓ] [] args)		

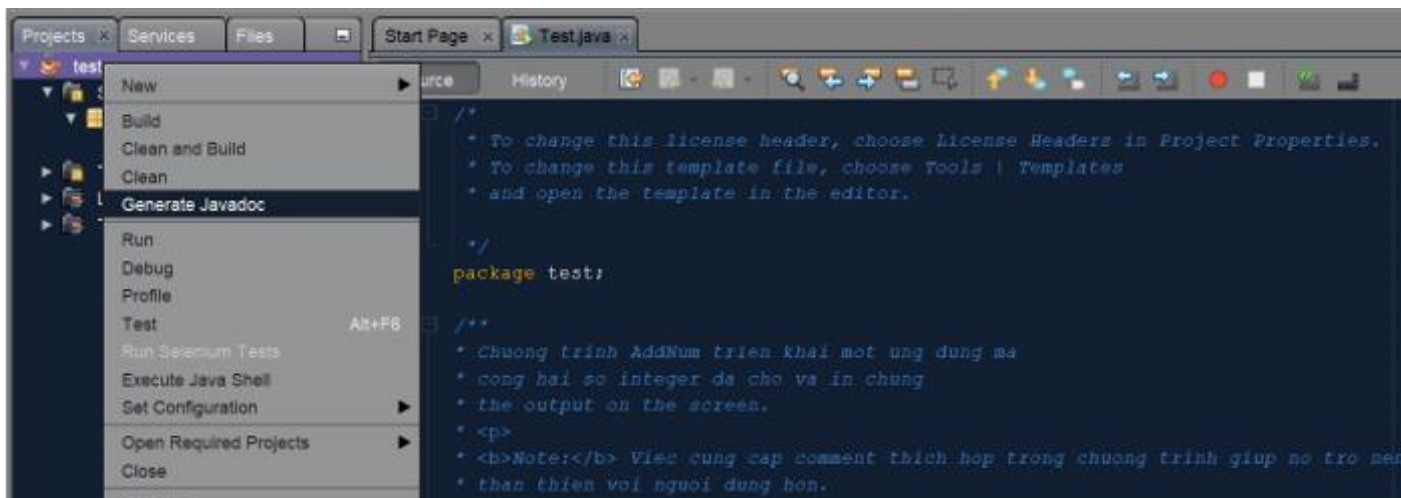
Methods inherited from class java.lang.Object[Ⓓ]

clone[Ⓓ], equals[Ⓓ], finalize[Ⓓ], getClass[Ⓓ], hashCode[Ⓓ], notify[Ⓓ], notifyAll[Ⓓ], toString[Ⓓ], wait[Ⓓ], wait[Ⓓ], wait[Ⓓ]



Công cụ lập tài liệu chương trình

- Tạo một HTML Document trên Netbeans:
 - Để tạo tài liệu cho một project đã viết:
 - B1: Chúng ta viết chú thích tiêu đề như hướng dẫn và ví dụ ở trên.
 - B2: Biên dịch Javadoc thành các trang HTML





Công cụ lập tài liệu chương trình

- B3: Kết quả sau khi biên dịch, tài liệu API do Javadoc sinh ra được lưu trong thư mục của dự án `../test/dist/javadoc`

Documents > NetBeansProjects > test > dist > javadoc

Name	Status	Date modified	Type	S
class-use	✓	2/16/2022 5:53 AM	File folder	
index-files	✓	2/16/2022 5:51 AM	File folder	
resources	✓	2/15/2022 2:43 PM	File folder	
script-dir	✓	2/15/2022 2:43 PM	File folder	
test	✓	2/15/2022 2:43 PM	File folder	
allclasses-index	✓	2/16/2022 5:58 AM	Microsoft Edge HTM...	
allpackages-index	✓	2/16/2022 5:58 AM	Microsoft Edge HTM...	
element-list	✓	2/16/2022 5:58 AM	File	
help-doc	✓	2/16/2022 5:58 AM	Microsoft Edge HTM...	
index	✓	2/16/2022 5:58 AM	Microsoft Edge HTM...	
jquery-ui.overrides	✓	2/16/2022 5:58 AM	Cascading Style Shee...	
member-search-index	✓	2/16/2022 5:58 AM	JavaScript File	
module-search-index	✓	2/16/2022 5:58 AM	JavaScript File	
overview-tree	✓	2/16/2022 5:58 AM	Microsoft Edge HTM...	
package-search-index	✓	2/16/2022 5:58 AM	JavaScript File	
package-summary	✓	2/16/2022 5:53 AM	Microsoft Edge HTM...	
package-tree	✓	2/16/2022 5:53 AM	Microsoft Edge HTM...	
package-use	✓	2/16/2022 5:53 AM	Microsoft Edge HTM...	
script	✓	2/16/2022 5:58 AM	JavaScript File	
search	✓	2/16/2022 5:58 AM	JavaScript File	
stylesheet	✓	2/16/2022 5:58 AM	Cascading Style Shee...	
tag-search-index	✓	2/16/2022 5:58 AM	JavaScript File	
Test	✓	2/16/2022 5:53 AM	Microsoft Edge HTM...	
type-search-index	✓	2/16/2022 5:58 AM	JavaScript File	



Nội dung

2. Công cụ và môi trường phát triển tích hợp

- Môi trường phát triển tích hợp
- Công cụ sinh mã nguồn từ thiết kế
- Công cụ quản lý mã nguồn
- Công cụ lập tài liệu chương trình

→ 3. Phong cách lập trình



Phong cách lập trình

- Phong cách lập trình (Programming Style)
 - Là quy tắc tối cơ bản về phong cách và quy tắc viết code.
- Tại sao lại cần có phong cách lập trình tốt?
 - Mục đích của một phong cách lập trình tốt là làm cho chương trình trở nên dễ đọc đối với người viết và người đọc chương trình.
 - Tránh các lỗi thường xảy ra do nhầm lẫn của lập trình viên:
 - Biến này được dùng làm gì?
 - Hàm này được gọi như thế nào?



Phong cách lập trình

- Làm thế nào để mã nguồn dễ đọc?
 - Cấu trúc chương trình rõ ràng, dễ hiểu, khúc triết.
 - Tổ chức chương trình theo modul
 - Quy tắc đặt tên biến, tên hàm.
 - Chọn tên phù hợp, gợi nhớ.
 - Quy tắc chú thích
 - Viết chú thích rõ ràng



Một số quy tắc

- **Nhất quán:**
 - Tuân thủ quy tắc đặt tên trong toàn bộ chương trình.
 - Nhất quán trong việc dùng các biến cục bộ.
- **Khúc triết:**
 - Mỗi chương trình con phải có một nhiệm vụ rõ ràng.
 - Đủ ngắn để có thể nắm bắt được.
 - Số tham số của chương trình con là tối thiểu (dưới 6)



Một số quy tắc

- Rõ ràng
 - Chú thích rõ ràng. Ví dụ: Đầu mỗi chương trình con.
- Bao đóng
 - Hàm chỉ nên tác động tới duy nhất 1 giá trị - giá trị trả về của hàm.
 - Không nên thay đổi giá trị của biến chạy trong thân của vòng lặp



Phong cách lập trình

- Tài liệu tham khảo:
 - https://users.soict.hust.edu.vn/trungtt/uploads/slides/KTLT_Bai06.pdf



Thảo luận

